

SkyChat API Documentation

Base URL: `https://nd1cgptf-3002.uksl.devttunnels.ms`

Table of Contents

1. [Overview](#)
2. [REST API Endpoints](#)
 - [2.1 Join / Create Room](#)
 - [2.2 Get Room Messages](#)
 - [2.3 Upload File](#)
 - [2.4 Add Message](#)
3. [WebSocket \(Socket.IO\) Events](#)
 - [3.1 Client → Server Events](#)
 - [3.2 Server → Client Events](#)
4. [Data Models](#)
5. [Full User Flow](#)
6. [Error Handling](#)

1. Overview

SkyChat is a real-time chat application with the following capabilities:

Feature	Implementation
Real-time messaging	Socket.IO
Room-based chat rooms	Server-managed rooms
Message persistence	MongoDB
File sharing (images, audio, video, documents)	Cloudinary
Voice recording	Client-side MediaRecorder + Cloudinary
Online/offline user status	MongoDB + Socket.IO
Typing indicators	Socket.IO

Key Technical Notes:

- All text messages and file metadata are persisted in MongoDB
- Files are uploaded to Cloudinary; only the URL + metadata is stored in the DB
- The server runs Socket.IO on the same port as HTTP (3002)
- Socket.IO uses namespace / (default)

2. REST API Endpoints

2.1 Join / Create Room

Registers a user in a room. If the user (name + room combination) already exists, returns the existing user.

```
POST {{base_url}}/go-to-room
Content-Type: application/json

{
  "name": "string",
  "room": "string"
}
```

Response 200 (new user created):

```
{
  "message": "done, user added",
  "data": {
    "id": "string",
    "name": "string",
    "room": "string",
    "isOnline": false
  }
}
```

Response 201 (user already existed):

```
{
  "message": "user existed",
  "data": {
    "id": "string",
    "name": "string",
    "room": "string",
    "isOnline": true
  }
}
```

Flutter Usage:

```
final response = await http.post(
  Uri.parse('$baseUrl/go-to-room'),
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({'name': username, 'room': roomName}),
);
```

2.2 Get Room Messages

Retrieves all messages for a given room, sorted chronologically (oldest first).

```
GET {{base_url}}/all-messages-for-room?room={room_name}
```

Response 200:

```
{
  "message": "chat found",
  "data": [
    {
      "id": "string",
      "content": "string",
      "room": "string",
      "userId": {
        "_id": "string",
        "name": "string",
        "room": "string",
        "isOnline": true
      },
      "file": {
        "url": "string",
        "publicId": "string",
        "type": "image | audio | video | file",
        "name": "string"
      },
      "createdAt": "ISO8601 DateTime"
    }
  ]
}
```

Note: `userId` is a **populated object** containing the sender's full user data, not just an ID. Each message also includes its MongoDB `id`.

Edge case: Messages with `userId: null` exist in the database (race condition when a user disconnects before the message is saved). Your mobile app should handle this gracefully — display them as "Unknown User" or filter them out.

2.3 Upload File

Uploads a file to Cloudinary and returns the URL and metadata. Supports images, audio, video, and documents.

```
POST {{base_url}}/upload-file
Content-Type: multipart/form-data

file: (binary)
```

Response 200:

```
{
  "url": "https://res.cloudinary.com/.../image/upload/v123456/file.png",
  "publicId": "skychat/abc123",
  "type": "image",
  "name": "original_filename.png"
}
```

type mapping based on MIME type:

MIME type	type value
image/*	image
audio/*	audio
video/*	video
anything else	file

⚠ Important: After uploading, you must **emit a `chatMessage` event via `Socket.IO`** with the file metadata to broadcast it to the room. The upload endpoint only stores the file — it does **not** send the message.

2.4 Add Message

(Optional — you can rely on `Socket.IO` for message creation instead.)

Saves a message directly to the database via REST.

```
POST {{base_url}}/add-message
Content-Type: application/json

{
  "name": "string",
  "room": "string",
  "content": "string",
  "file": {
    "url": "string",
    "publicId": "string",
    "type": "image | audio | video | file",
    "name": "string"
  }
}
```

Response 200:

```
{
  "message": "message write done",
  "data": {
    "id": "string",
    "content": "string",
    "room": "string",
    "userId": "string",
    "file": { ... },
    "createdAt": "ISO8601 DateTime"
  }
}
```

Note: The `userId` field here is a plain string ID (not populated). Messages created via Socket.IO (`chatMessage` event) are also persisted automatically — this endpoint is a fallback

3. WebSocket (Socket.IO) Events

Socket.IO client library: `socket.io-client` (Dart: https://pub.dev/packages/socket_io_client)

3.1 Client → Server Events

`joinRoom`

Emit when the user enters a chat room. The server will:

1. Register the user in the in-memory store
2. Join the Socket.IO room
3. Set `isOnline = true` in MongoDB
4. Send previous messages (via `joinedRoom` event)
5. Notify all room members

```
socket.emit('joinRoom', {
  'userName': 'John',
  'room': 'General',
});
```

`chatMessage`

Emit to send a text message or a file message.

Plain text:

```
socket.emit('chatMessage', 'Hello everyone!');
```

With file attachment:

```
socket.emit('chatMessage', {
  'text': 'Check this out',
  'file': {
    'url': 'https://res.cloudinary.com/...',
    'publicId': 'skychat/abc123',
    'type': 'image',
    'name': 'photo.jpg',
  },
});
```

 The file metadata must be obtained first via the `/upload-file` REST endpoint. The Socket.IO event only **broadcasts** the metadata — the actual file is already on Cloudinary.

`typing`

Broadcasts typing status to other users in the room.

```
// Start typing
socket.emit('typing', true);

// Stop typing
socket.emit('typing', false);
```

3.2 Server → Client Events

joinedRoom

Sent to the connecting user after they successfully join a room.

```
socket.on('joinedRoom', (data) {
  print('Joined room: ${data['room']}');
});
```

message

Received when a new message is sent in the room (including your own messages).

```
socket.on('message', (data) {
  final userName = data['userName']; // String
  final text = data['text'];         // String
  final time = data['time'];          // String (HH:MM format)
  final file = data['file'];          // Map or null

  // File structure (when present):
  // file['url']      -> String
  // file['publicId'] -> String
  // file['type']     -> String ("image" | "audio" | "video" | "file")
  // file['name']     -> String
});
```

Special senders:

- `userName == "Chat"` — system messages (join/leave notifications, welcome messages). Display differently (centered, muted styling).

roomUsers

Received when the user list in the room changes (user joins or leaves).

```
socket.on('roomUsers', (data) {
  final room = data['room']; // String
  final users = data['users'] as List; // List of user objects

  // Each user object:
  // user['id']      -> String
  // user['name']    -> String
  // user['room']    -> String
  // user['isOnline'] -> bool
});
```

displayTyping

Received when another user in the room starts or stops typing.

```
socket.on('displayTyping', (data) {
  final userName = data['userName']; // String
  final isTyping = data['isTyping']; // bool
});
```

4. Data Models

User

```
{
  "id": "string",
  "name": "string",
  "room": "string",
  "isOnline": false
}
```

Message

```
{
  "id": "string",
  "content": "string",
  "room": "string",
  "userId": "string | UserObject",
  "file": {
    "url": "string",
    "publicId": "string",
    "type": "image | audio | video | file",
    "name": "string"
  },
  "createdAt": "ISO8601 DateTime"
}
```

userId is a string ID when returned from /add-message or createMessage. It is a populated UserObject when returned from /all-messages-for-room (mongoose .populate()).

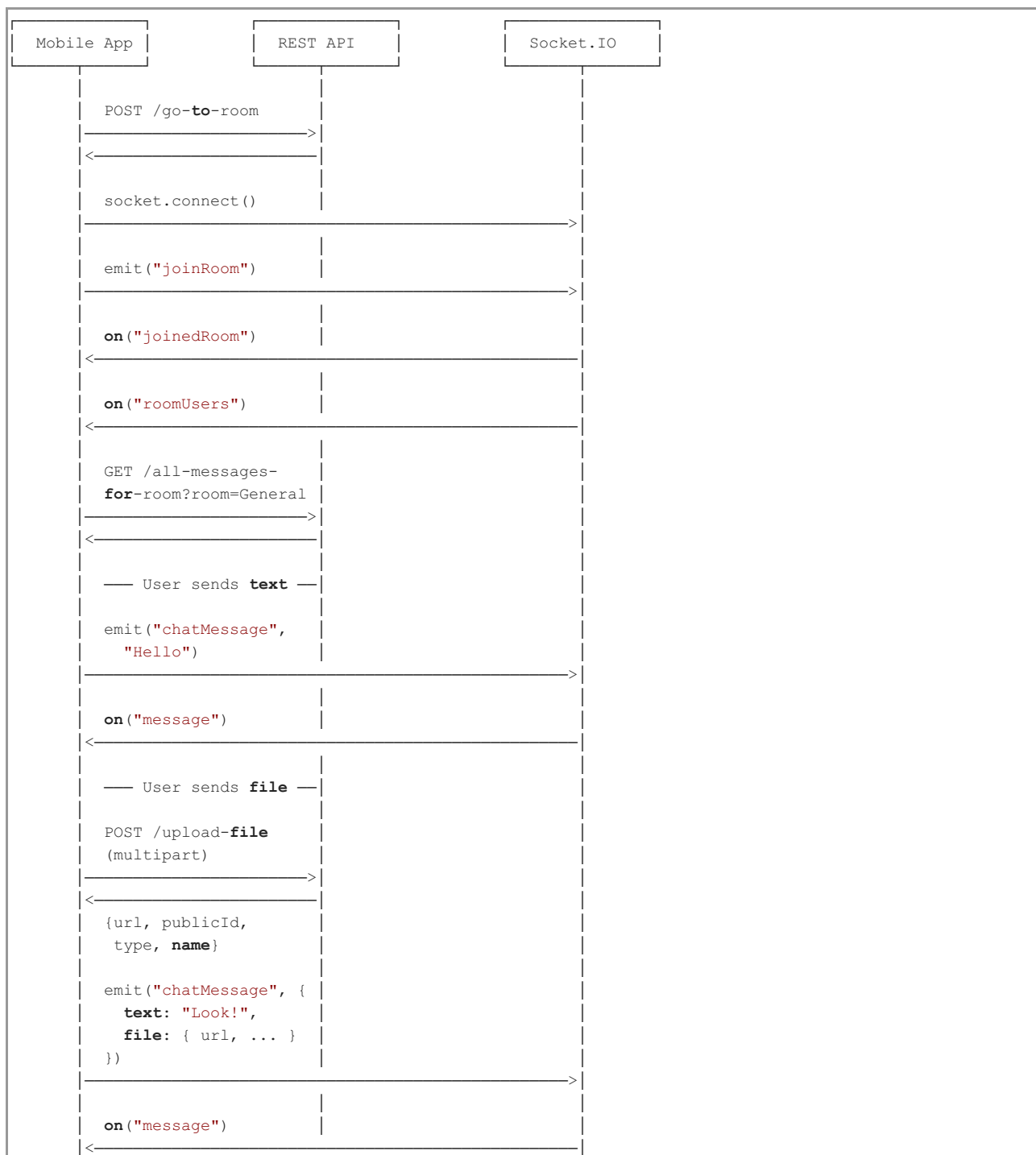
FileAttachment

```
{
  "url": "string",
  "publicId": "string",
  "type": "image | audio | video | file",
  "name": "string"
}
```

Socket Message (from message event)

```
{
  "userName": "string",
  "text": "string",
  "time": "string (HH:MM format)",
  "file": {
    "url": "string",
    "publicId": "string",
    "type": "image | audio | video | file",
    "name": "string"
  }
}
```

5. Full User Flow



6. Error Handling

HTTP Status Codes

Code	Meaning
200	Success
201	Success — user already existed
400	Bad request — missing or invalid parameters
404	User not found (usually on <code>/add-message</code>)
500	Server error — upload failed, DB error

Common Issues

1. Messages with `userId: null`

These occur when a user disconnects before their chat message is saved to MongoDB. The message still exists but has no associated user. Handle in the app:

```
final senderName = msg['userId']?['name'] ?? 'Unknown User';
```

2. File upload fails

Cloudinary upload may fail for very large files or unsupported formats. The server returns 500 with {"message": "upload failed"}.

3. Socket disconnection

Implement Socket.IO reconnection handling:

```
socket.on('disconnect', (_) {  
  // Show reconnection overlay  
});  
socket.on('reconnect', (_) {  
  // Re-join room and refresh messages  
});
```